

定义变量:

```
do {
    // ...
    mumble(foo);
} while (int foo = get_foo()); // 错误: 将变量声明放在了 do 的条件部分
```

如果允许在条件部分定义变量, 则变量的使用出现在定义之前, 这显然是不合常理的!

190

5.4.4 节练习

练习 5.18: 说明下列循环的含义并改正其中的错误。

```
(a) do
    int v1, v2;
    cout << "Please enter two numbers to sum:" ;
    if (cin >> v1 >> v2)
        cout << "Sum is: " << v1 + v2 << endl;
    while (cin);
(b) do {
    // ...
} while (int ival = get_response());
(c) do {
    int ival = get_response();
} while (ival);
```

练习 5.19: 编写一段程序, 使用 do while 循环重复地执行下述任务: 首先提示用户输入两个 string 对象, 然后挑出较短的那个并输出它。

5.5 跳转语句

跳转语句中断当前的执行过程。C++语言提供了 4 种跳转语句: break、continue、goto 和 return。本章介绍前三种跳转语句, return 语句将在 6.3 节 (第 199 页) 进行介绍。

5.5.1 break 语句

break 语句 (break statement) 负责终止离它最近的 while、do while、for 或 switch 语句, 并从这些语句之后的第一条语句开始继续执行。

break 语句只能出现在迭代语句或者 switch 语句内部 (包括嵌套在此类循环里的语句或块的内部)。break 语句的作用范围仅限于最近的循环或者 switch:

```
string buf;
while (cin >> buf && !buf.empty()) {
    switch(buf[0]) {
        case '-':
            // 处理到第一个空白为止
            for (auto it = buf.begin()+1; it != buf.end(); ++it) {
                if (*it == ' ')
                    break; // #1, 离开 for 循环
            }
            // ...
    }
```

```
    }  
    // break #1 将控制权转移到这里  
    // 剩余的 '-' 处理:  
    break; // #2, 离开 switch 语句  
    case '+':  
        //...  
    } // 结束 switch  
    // 结束 switch: break #2 将控制权转移到这里  
} // 结束 while
```

191

标记为#1 的 `break` 语句负责终止连字符 `case` 标签后面的 `for` 循环。它不但不会终止 `switch` 语句，甚至连当前的 `case` 分支也终止不了。接下来，程序继续执行 `for` 循环之后的第一条语句，这条语句可能接着处理连字符的情况，也可能是另一条用于终止当前分支的 `break` 语句。

标记为#2 的 `break` 语句负责终止 `switch` 语句，但是不能终止 `while` 循环。执行完这个 `break` 后，程序继续执行 `while` 的条件部分。

5.5.1 节练习

练习 5.20: 编写一段程序，从标准输入中读取 `string` 对象的序列直到连续出现两个相同的单词或者所有单词都读完为止。使用 `while` 循环一次读取一个单词，当一个单词连续出现两次时使用 `break` 语句终止循环。输出连续重复出现的单词，或者输出一个消息说明没有任何单词是连续重复出现的。

5.5.2 `continue` 语句

`continue` 语句 (`continue statement`) 终止最近的循环中的当前迭代并立即开始下一次迭代。`continue` 语句只能出现在 `for`、`while` 和 `do while` 循环的内部，或者嵌套在此类循环里的语句或块的内部。和 `break` 语句类似的是，出现在嵌套循环中的 `continue` 语句也仅作用于离它最近的循环。和 `break` 语句不同的是，只有当 `switch` 语句嵌套在迭代语句内部时，才能在 `switch` 里使用 `continue`。

`continue` 语句中断当前的迭代，但是仍然继续执行循环。对于 `while` 或者 `do while` 语句来说，继续判断条件的值；对于传统的 `for` 循环来说，继续执行 `for` 语句头的 *expression*；而对于范围 `for` 语句来说，则是用序列中的下一个元素初始化循环控制变量。

例如，下面的程序每次从标准输入中读取一个单词。循环只对那些以下画线开头的单词感兴趣，其他情况下，我们直接终止当前的迭代并获取下一个单词：

```
string buf;  
while (cin >> buf && !buf.empty()) {  
    if (buf[0] != '_')  
        continue; // 接着读取下一个输入  
    // 程序执行过程到了这里？说明当前的输入是以下画线开始的；接着处理 buf.....  
}
```

5.5.2 节练习

192

练习 5.21: 修改 5.5.1 节（第 171 页）练习题的程序，使其找到的重复单词必须以大写字母开头。

5.5.3 goto 语句

goto 语句 (goto statement) 的作用是从 goto 语句无条件跳转到同一函数内的另一条语句。



不要在程序中使用 goto 语句，因为它使得程序既难理解又难修改。

goto 语句的语法形式是

```
goto label;
```

其中，*label* 是用于标识一条语句的标示符。带标签语句 (labeled statement) 是一种特殊的语句，在它之前有一个标示符以及一个冒号：

```
end: return; // 带标签语句，可以作为 goto 的目标
```

标签标示符独立于变量或其他标示符的名字，因此，标签标示符可以和程序中其他实体的标示符使用同一个名字而不会相互干扰。goto 语句和控制权转向的那条带标签的语句必须位于同一个函数之内。

和 switch 语句类似，goto 语句也不能将程序的控制权从变量的作用域之外转移到作用域之内：

```
// ...
goto end;
int ix = 10; // 错误：goto 语句绕过了一个带初始化的变量定义
end:
// 错误：此处的代码需要使用 ix，但是 goto 语句绕过了它的声明
ix = 42;
```

向后跳过一个已经执行的定义是合法的。跳回到变量定义之前意味着系统将销毁该变量，然后重新创建它：

```
// 向后跳过一个带初始化的变量定义是合法的
begin:
    int sz = get_size();
    if (sz <= 0) {
        goto begin;
    }
```

在上面的代码中，goto 语句执行后将销毁 *sz*。因为跳回到 *begin* 的动作跨过了 *sz* 的定义语句，所以 *sz* 将重新定义并初始化。

193

5.5.3 节练习

练习 5.22：本节的最后一个例子跳回到 *begin*，其实使用循环能更好地完成该任务。重写这段代码，注意不再使用 goto 语句。

5.6 try 语句块和异常处理

异常是指存在于运行时的反常行为，这些行为超出了函数正常功能的范围。典型的异常包括失去数据库连接以及遇到意外输入等。处理反常行为可能是设计所有系统最难的一部分。